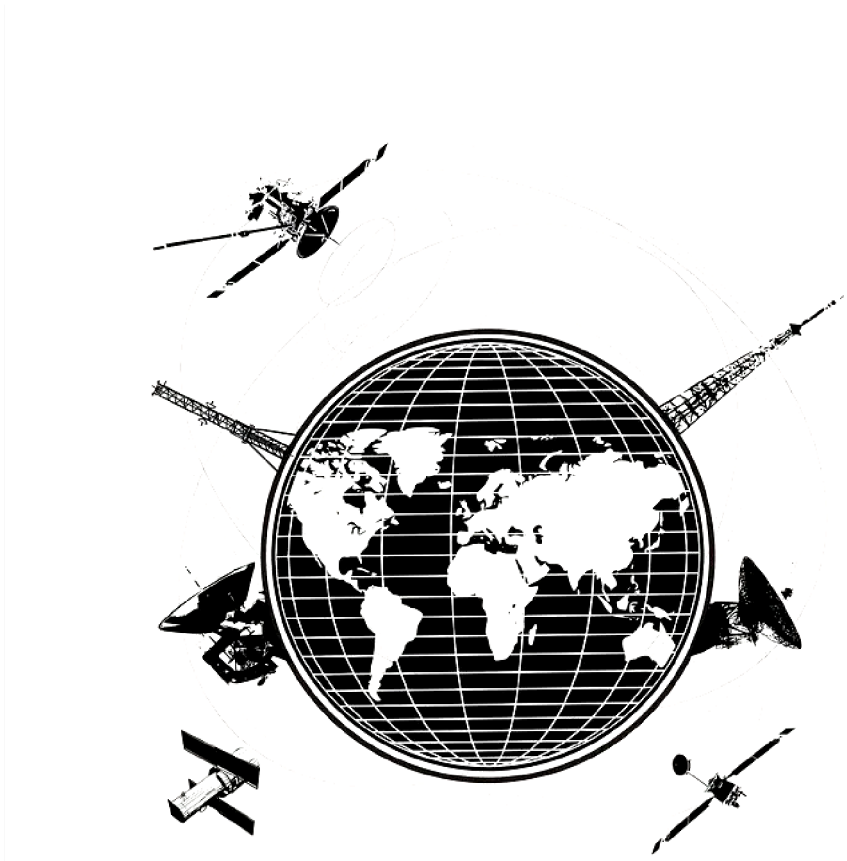
slack engineering

APRIL 11, 2023 8 MIN READ

Real-time Messaging



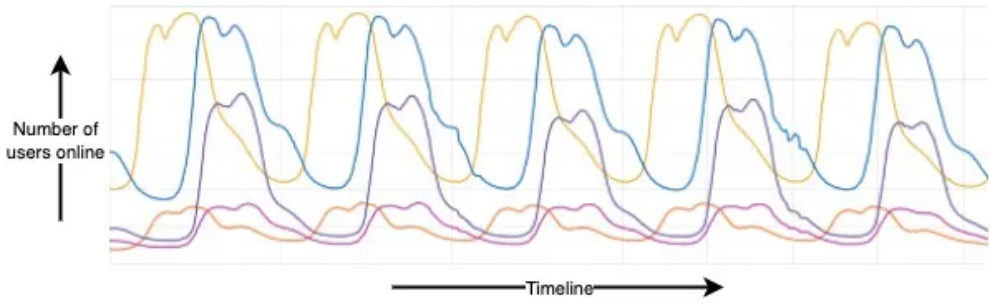
Sameera Thangudu



Did you know that ground stations transmit signals to satellites 22,236 miles above the equator in

geostationary orbits, and that those signals are then beamed down to the entire North American subcontinent? Satellite radios today serve hundreds of channels across 9,540,000 square miles. Unless you're working at a secret military facility, deep underground, you can enjoy satellite radio everywhere.

Just like the satellites, Slack sends millions of messages every day across millions of channels in real time all around the world. If we look at the traffic on a typical work day, it shows that most users are online between 9am and 5pm local time, with peaks at 11am and 2pm and a small dip in between for lunch hour. Though the working hours are similar across regions, looking at the two peaks in the graph below, it is evident that prime time is not the same: It's post-noon in some regions and pre-noon in other regions. Each colored line in the below graph represents a region.



In this blog post we'll describe the architecture that we use to send real-time messages at this scale. We'll

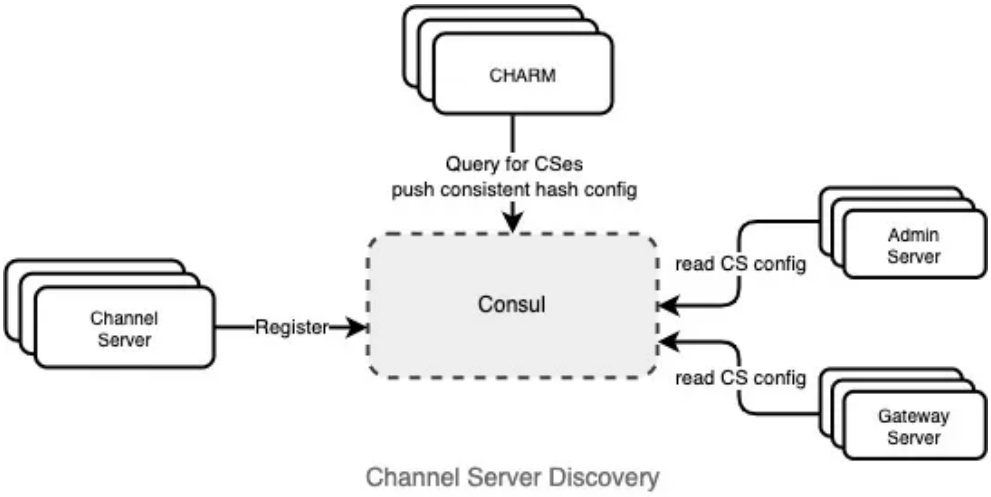
take a closer look at the services that send the chat messages and various events to these online users in real time. Our core services are written in Java: They are Channel Servers, Gateway Servers, Admin Servers, and Presence Servers.

Server overview

Channel Servers (CS) are stateful and in-memory, holding some amount of history of channels. Every CS is mapped to a subset of channels based on consistent hashing. At peak times, about 16 million channels are served per host. A “channel” in this instance is an abstract term whose ID is assigned to an entity such as user, team, enterprise, file, huddle, or a regular Slack channel. The ID of the channel is hashed and mapped to a unique server. Every CS host receives and sends messages for those mapped channels. A single Slack team has all of its channels mapped across all the CSs.

Consistent hash ring managers (CHARMs) manage the consistent hash ring for CSs. They replace unhealthy CSs very quickly and efficiently; a new CS is ready to serve traffic in under 20 seconds. With a team’s channels spread across all CSs, a small number of teams’ channels are mapped to a CS. When a channel server is replaced, users of those teams’ channels experience elevated latency in message delivery for less than 20 seconds.

The diagram below shows how CSs are registered in Consul, our service discovery tool. Each consistent hash is defined and managed by CHARMs, and then Admin Servers (AS) and CS discovers them by querying Consul for the up-to-date config.



Channel Server Discovery

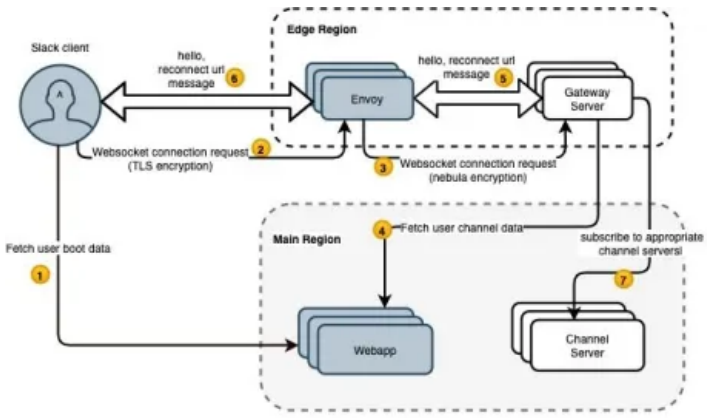
Gateway Servers (GS) are stateful and in-memory. They hold users' information and websocket channel subscriptions. This service is the interface between Slack clients and CSs. Unlike all other servers, GSs are deployed across multiple geographical regions. This allows a Slack client to quickly connect to a GS host in its nearest region. We have a draining mechanism for region failures that seamlessly switches the users in a bad region to the nearest good region.

Admin Servers (AS) are stateless and in-memory. They interface between our Webapp backend and CSs. Presence Servers (PS) are in-memory and keep

track of which users are online. It powers the green presence dots in Slack clients. The users are hashed to individual PSs. Slack clients make queries to it through the websocket using the GS as a proxy for presence status and presence change notifications. A Slack client receives presence notifications only for a subset of users that are visible in the app screen at any moment.

Slack client set up

Every Slack client has a persistent websocket connection to Slack's servers to receive real-time events to maintain its state. The client sets up a websocket connection as below.



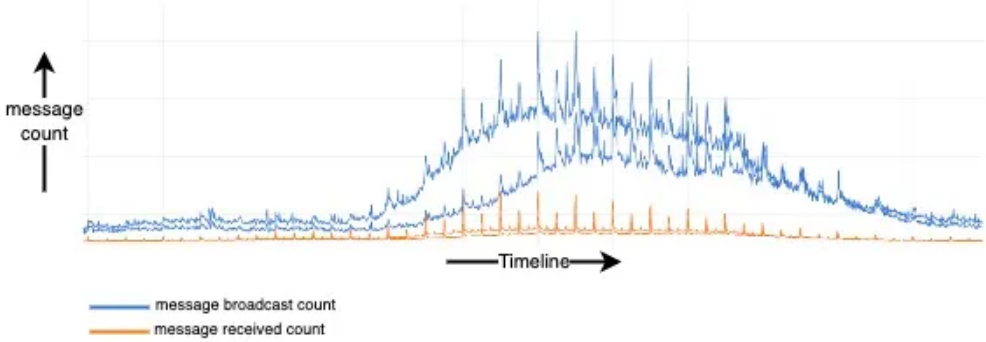
Slack Client Websocket set up

On boot up, the client fetches the user token and websocket connection setup information from the Webapp backend. Webapp is a Hacklang codebase

that hosts all the APIs called by our Slack Clients. This service also includes JavaScript code that renders the Slack clients. A client initiates a websocket connection to the nearest edge region. Envoy forwards the request to GS. Envoy is an open source edge and service proxy, designed for cloud-native applications. Envoy is used at Slack as a load-balancing solution for various services and TLS termination. GS fetches the user information, including all the user's channels, from Webapp and sends the first message to the client. GS then subscribes to all the channel servers that hold those channels based on consistent hashing asynchronously. The Slack client is now ready to send and receive real time messages.

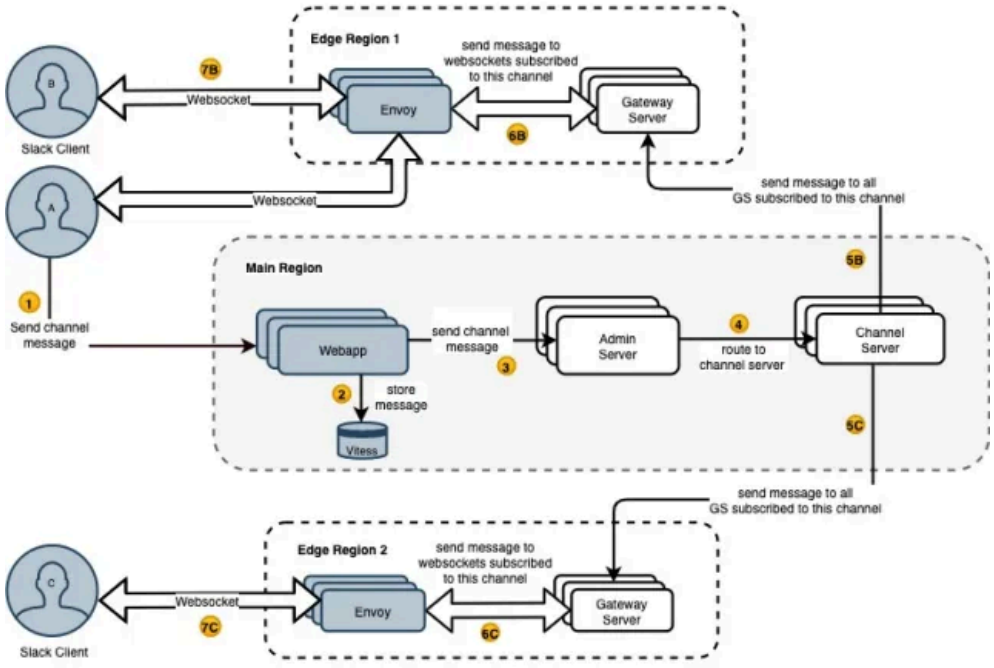
Send a message to a million clients in real time

Once the client is set up, each message sent in a channel is broadcasted to all clients online in the channel. Our message stats shows that the multiplicative factor for message broadcast is different across regions, with some regions having a higher rate than others. This could be due to multiple factors, including team sizes in those regions. The chart below shows message received count and message broadcasted count across multiple regions.



Let’s take a look at how the message is broadcasted to all online clients. Once the websocket is set up, as discussed above, the client hits our Webapp API to send a message. Webapp then sends that message to AS. AS looks at the channel ID in this message, discovers CS through a consistent hash ring, and routes the message to the appropriate CS that hosts the real time messaging for this channel. When CS receives the message for that channel, it sends out the message to every GS across the world that is subscribed to that channel. Each GS that receives that message sends it to every connected client subscribed to that channel id.

Below is a journey of a message from the client through our stack. In the following example, Slack client A and B are in the same edge region, and C is in a different region. Client A is sending a message, and client B and C are receiving it.

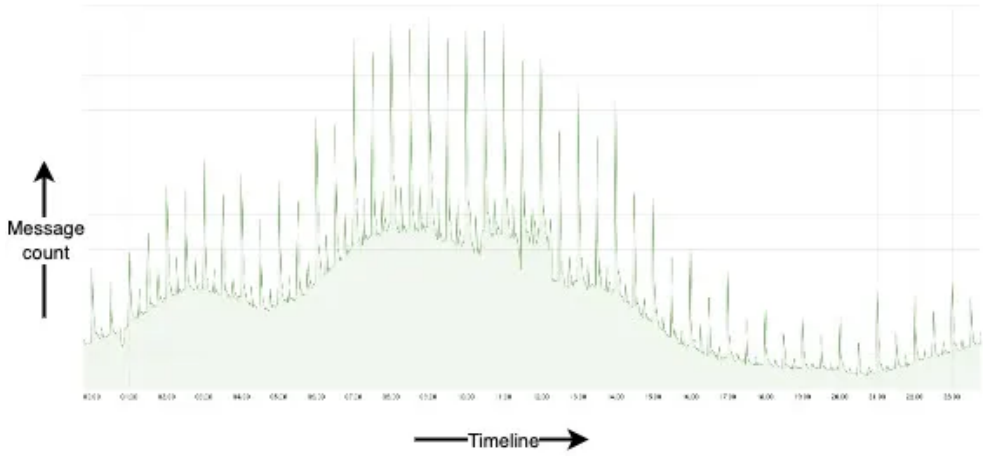


Journey of a message

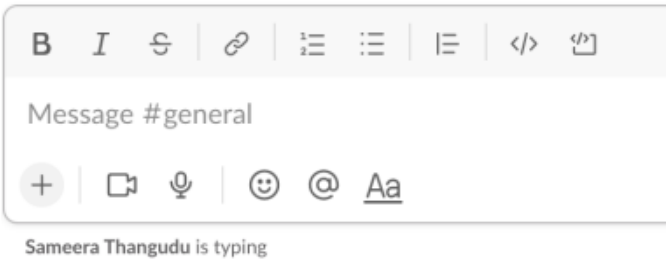
Events

Aside from chat messages, there is another special kind of message called an event. An event is any update a client receives in real time that changes the state of the client. There are hundreds of different types of events that flow across our servers. Some examples include when a user sends a reaction to a message, a bookmark is added, or a member joins a channel. These events follow a similar journey to the simple chat message shown above.

Look at the message delivery graph below. The count spikes at regular intervals. What could cause these spikes? Turns out, events sent for reminders, scheduled messages, and calendar events tend to happen at the top of the hour, explaining the regular traffic spikes.

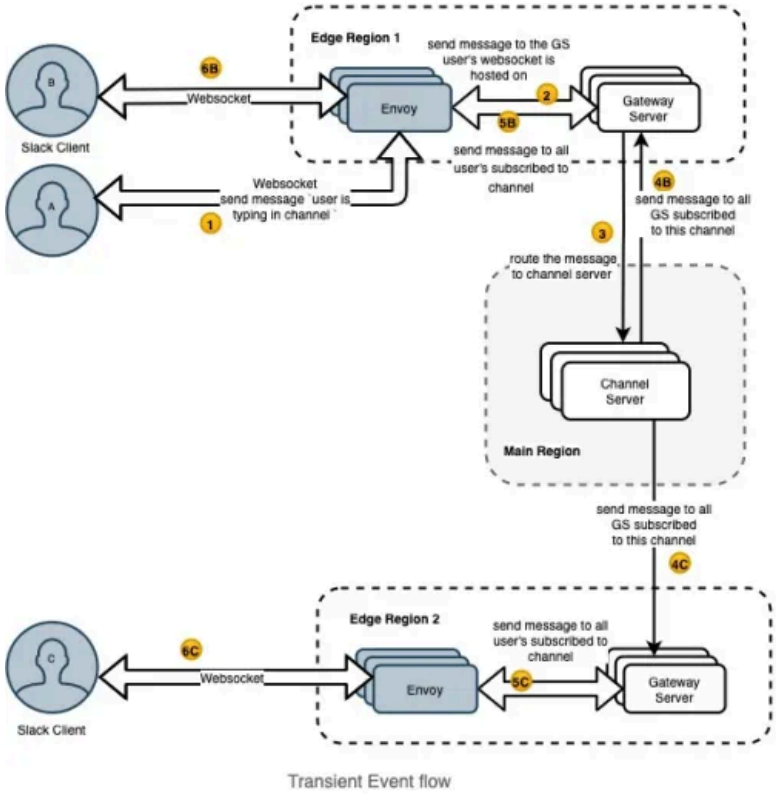


Now let's take a look at a different kind of event called Transient events. These are a category of events that are not persisted in the database and are sent through a slightly different flow. User typing in a channel or a document is one such event.



Below is a diagram that shows this scenario. Again, Slack client A and B are in the same edge region, and C is in a different region. Slack client A is typing in a channel and this is notified to other users B and C in the channel. Client A sends this message via websocket to GS. GS looks at the channel ID in the

message and routes to the appropriate CS based on a consistent hash ring. CS then sends to all GSs across the world subscribed to this channel. Each GS, on receiving this message, broadcasts to all the users websockets subscribed to this channel



Transient Event flow

What's next

Our servers serve tens of millions of channels per host, tens of millions of connected clients, and our system delivers messages across the world in 500ms. With the linear scalability of our current architecture, our projections show that we can serve many more customers. However, there is always room for

improvement and we are looking to extend our architecture to serve the scale of our next biggest customers. If this work sounds interesting to you, come join us: we have an open role !

Lastly, a huge shout out to everyone who contributed to this architecture, and to Serguei Mourachov for reviewing and giving feedback on this blog post.

#infrastructure #scalability #software-architecture

SEARCH

LATEST POSTS

JULY 14, 2026 15 MIN READ

Shipyard: How We Built Slack's Next-Generation EC2 Platform

JUNE 11, 2026 12 MIN READ

Agentic Testing: Where Agents Fit in the E2E Testing Stack

MAY 28, 2026 17 MIN READ

Slack AI: The Path to Multi-Cloud

MAY 5, 2026 15 MIN READ

From SSH to REST: A Security-Driven Modernization of Slack's EMR Data Pipelines

APRIL 13, 2026 14 MIN READ

Managing context in long-run agentic applications

ARCHIVES

2026	(7)
2025	(9)
2024	(19)
2023	(16)
2022	(21)
2021	(24)
2020	(26)
2019	(21)
2018	(11)
2017	(21)
2016	(19)

PREVIOUS POST

Tracing Notifications

Notifications are a key aspect of the Slack user experience. Users rely on timely notifications…

APRIL 4, 2023 12 MIN READ



NEXT POST

Service Delivery Index: A Driver for Reliability

Customer-first: Moving from Hero Engineering to Reliability Engineering From the beginning, Slack has always had…

JULY 26, 2023 10 MIN READ



Recommended Reading



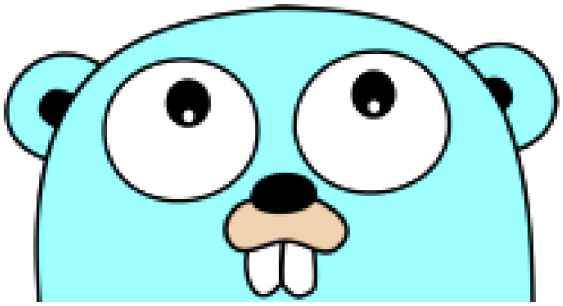
MAY 28, 2026 17 MIN READ

Slack AI: The Path to Multi-Cloud

@Shaurya Kethireddy







MARCH 31, 2026 5 MIN READ

From Custom to Open: Scalable Network Probing and HTTP/3 Readiness with Prometheus

@Slack Engineering



OCTOBER 7, 2025 11 MIN READ

Deploy Safety: Reducing customer impact from change

@Sam Bailey



OCTOBER 10, 2024 9 MIN READ

We're All Just Looking for Connection

@Brett Wines

slack engineering

ARTICLES CAREERS SLACK DEVELOPER BLOG THE SLACK BLOG



Terms of Service Privacy Information Cookie Preferences Your Privacy Choices

© 2026 Slack Technologies, LLC, a Salesforce company. All rights reserved. Various trademarks held by their respective owners.